

"Opossum Attack: Application Layer Desynchronization using Opportunistic TLS"

in USENIX Security'26 분석

이동훈

dhsama51@gmail.com

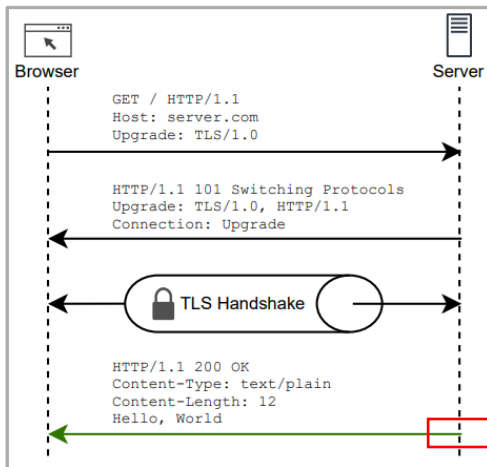
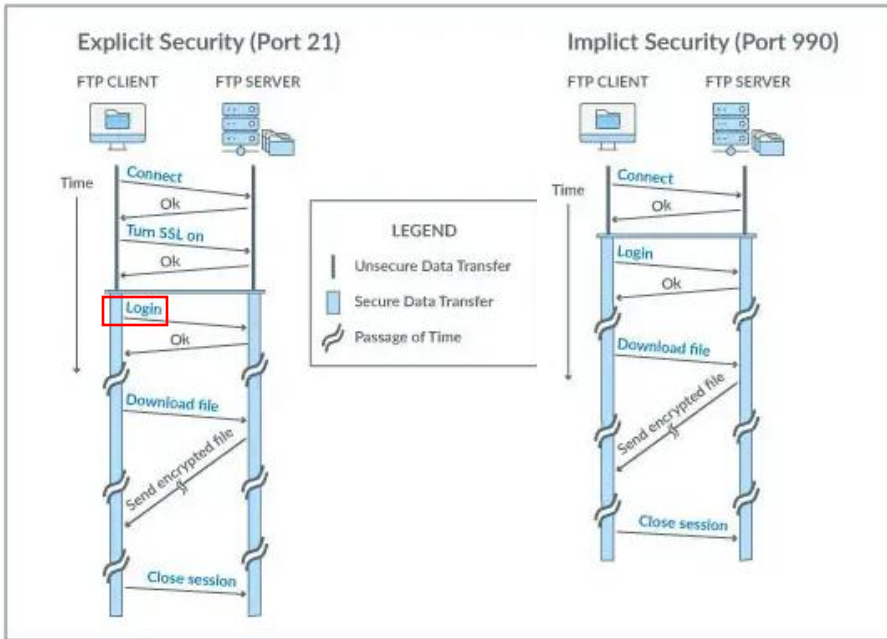
2026.09.07

Contents

1. Background - Implicit / Opportunistic TLS & ALPACA Attack
2. Attack Scenarios - I2O / O2I Desynchronization
3. Opossum Attack Structure
4. HTTPS Desynchronization Attacks
5. Experiments and Evaluation - IPv4-wide Scan
6. Existing Defense Mechanisms
7. Challenges
8. Contributions
9. Limitations
10. More Interested Papers (in IEEE S&P'26, USENIX'26)

1. Background - Implicit vs Opportunistic TLS

- 두 가지 TLS 통합 방식



- Implicit TLS: 새 Port에서 TLS handshake → 시작부터 TLS 통신
ex. HTTPS(443)
- Opportunistic TLS: 기존 Port · 평문 통신 → STRTTLS 등으로 TLS 전환
설계 철학: "일부라도 보호하는 게 낫다"
ex. HTTP(80)
- HTTP·FTP·SMTP·IMAP·POP3·LDAP 등 다수가 opportunistic TLS를 지원
→ protocol마다 TLS 이후 먼저 메시지를 보내는 대상이 다른
엄밀히는 implicit TLS와 다른 protocol
- Synchronous Protocol은 응답이 요청과 같은 순서로 온다고 가정
→ Attacker가 순서를 바꾸어도 알아채지 못함
ex. HTTP 1.0/1.1, SMTP, POP3, FTP

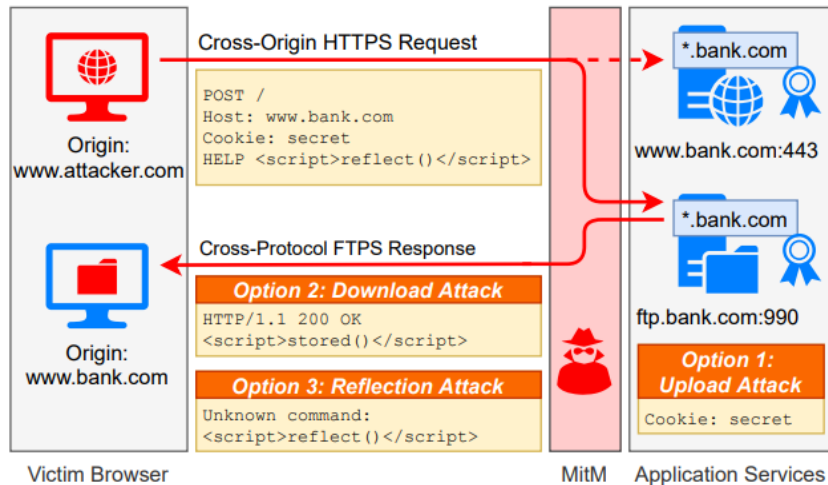
1. Background - ALPACA Attack

- Related Work: ALPACA (USENIX Sec'21)

- 핵심 Exploit: 동일 인증서를 쓰는 다른 protocol(ex. HTTPS ↔ FTPS)로 트래픽을 우회시켜, cross-protocol로 통신하게 만들

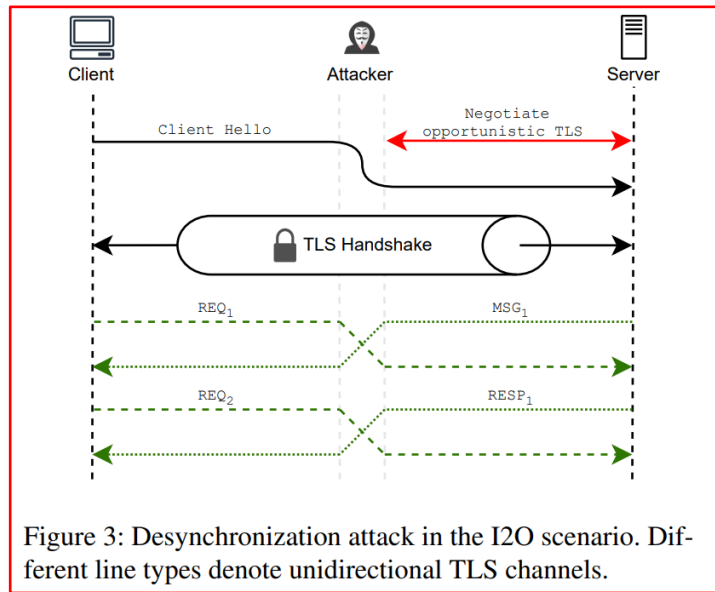
- TLS가 기밀성, 무결성, server domain 인증을 보장하지만, TLS 인증서가 **protocol을 구분해주지 않음**
- HTTP는 **web origin <protocol, IP, port>**를 통해 식별하는데, TLS는 인증서로는 protocol을 보장할 수 없음

ex. "Browser→Server HTTPS ClientHello"를 MitM으로 FTPS로 전달하게 만들면, Server가 이를 인식하지 못하고 TLS handshake 정상 진행



- 대응책(RFC 9325): strict ALPN / SNI 검증 → 서버가 "클라이언트와 같은 응용 프로토콜"을 쓰는지 확인 가능
 - opportunistic TLS와 implicit TLS는 **동일한 ALPN 식별자**를 사용하여 우회 가능
 - handshake 이후 먼저 메시지를 보내는 side가 다른 protocol은 desynchronization이 가능함

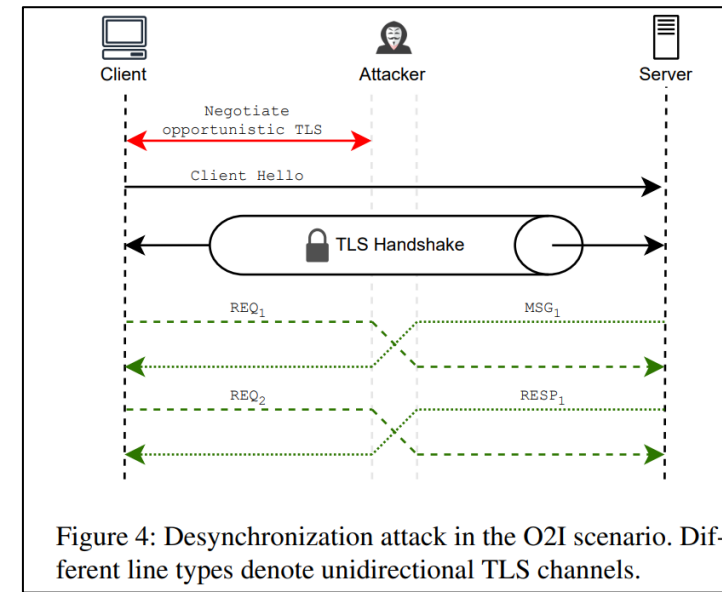
2. Attack Scenarios - I2O / O2I Desynchronization



Protocol	TLS		Impact
	Implicit	Opportunistic	
FTP	↗	↘	! O2I
HTTP 1.1	↘	↗	⚡ I2O
IMAP	↗	↘	(b) O2I
IRC	↘	↘	
LDAP	↘	↘	
LMTP	↗	↘	! O2I
Managed Sieve	↗	↗	
NNTP	↗	↘	! O2I
POP3	↗	↘	! O2I
PostgreSQL	↘	↘	
RDP	↘	↘	
SMTP	↗	↘	! O2I
MySQL		↘	
NBD		↘	
XMPP		↘	

! Affected ⚡ Exploit shown
 ↗ First AppData from client ↘ First AppData from server
 O2I Opportunistic to implicit scenario
 I2O Implicit to opportunistic scenario

Table 1: Summary of our evaluation of various protocols in the context of the Opossum attack. The last column indicates



I2O (Client: Implicit → Server: Opportunistic) → 대상: HTTP 1.1

- ① 공격자가 ClientHello를 delay시키고, 서버에 opportunistic upgrade 요청 → 이때 원하는 메시지를 삽입할 수 있음
 - ② Client의 ClientHello를 Server에 전송하면서 TLS handshake 진행
 - ③ Client는 REQ1 전송, Server는 공격자의 요청에 대한 응답(MSG1) 전송 → 재배치하면서 MSG1이 REQ1의 응답인 것처럼 Client에 도착
- 이후 모든 응답이 한 칸씩 밀려 desynchronize 됨

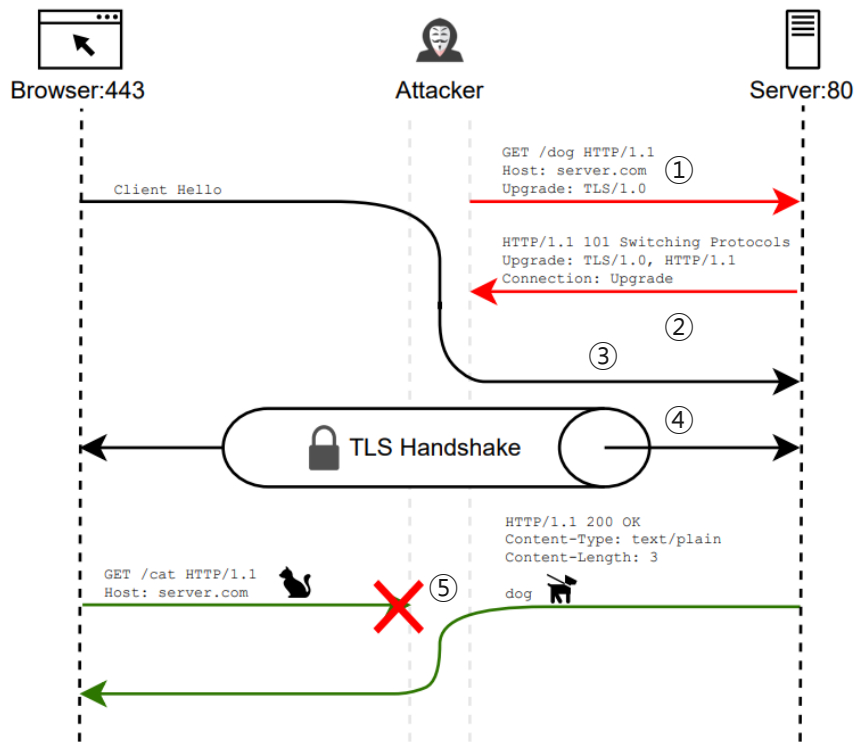
O2I (Client: Opportunistic → Server: Implicit) → 대상: FTP, SMTP, IMAP, POP3, LDAP

- ① 공격자가 Client에 평문으로 환영 메시지와 함께 opportunistic upgrade 요청
 - ② Client의 ClientHello를 Server에 전송하면서 TLS handshake 진행
 - ③ Client는 REQ1 전송, Server는 환영 메시지(MSG1) 전송 → 재배치하면서 MSG1이 REQ1 응답인 것처럼 Client에 도착
- 이후 모든 응답이 한 칸씩 밀려 desynchronize 됨

HTTP: 공격자가 **임의의 평문 요청**을 주입 가능 → 서버가 그 요청에 대한 응답 내용까지 통제 가능

O2I 대상 프로토콜: 주입 가능한 메시지가 **환영 메시지 정도로 제한적** → 공격 활용도가 HTTP보다 훨씬 낮음 → PoC만 존재

3. Opossum Attack Structure



- ① Browser의 ClientHello를 delay시키고, 공격자가 서버에 GET /dog (임의의 요청) + Upgrade: TLS 요청 (opportunistic 개시) 전송
- ② 서버는 101 Switching Protocols로 업그레이드 수락
- ③ 공격자가 지연시켰던 Browser의 Client Hello를 그대로 서버에 전달
TLS handshake가 Browser ↔ Server 간 정상 수행
→ Browser는 implicit으로, Server는 opportunistic으로 생각함
- ④ Browser는 GET /cat 전송, Server는 대기 중이던 /dog 응답을 전송
→ Attacker가 cat 요청을 drop하고, Browser의 dog 응답을 forward
→ cat 요청에 대한 응답으로 오인

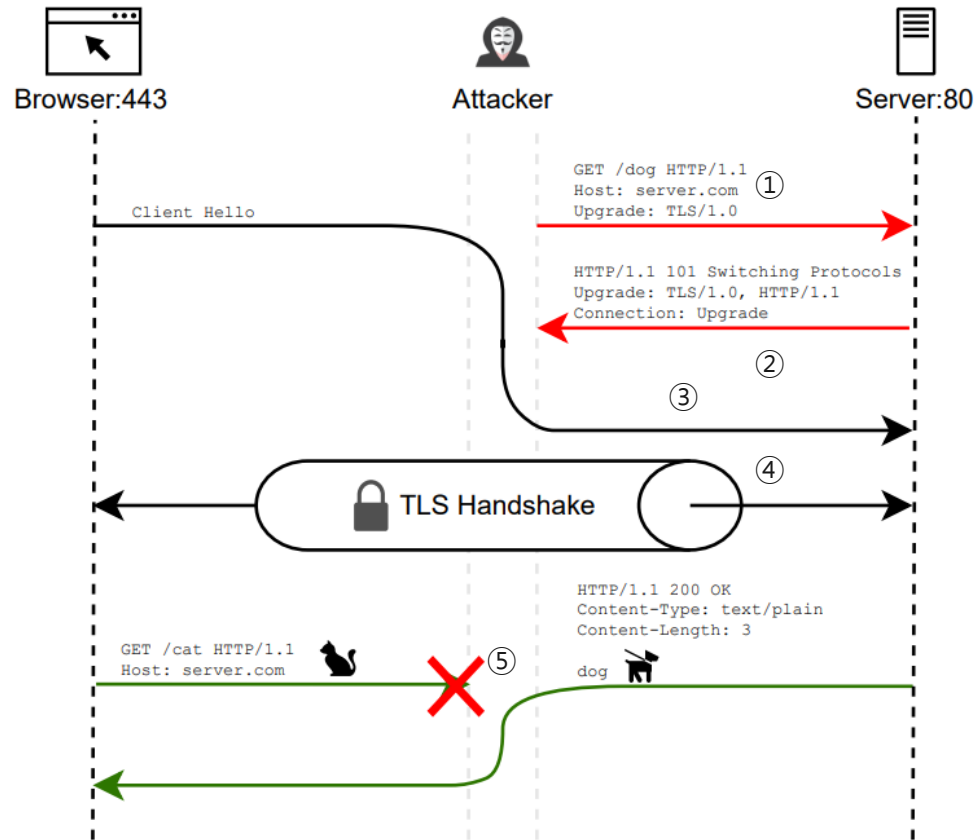
이후에도 desynchronization은 유지됨

(MitM Attacker (on-path, 순수 중계자) 가정)

Client, Server는 표준을 준수한다고 가정

- 인증서는 Server domain을 인증함에도 opportunistic / implicit TLS를 구분하지 않아 desynchronization이 가능
- HTTPS 기준 handshake 후 implicit은 Client가 먼저 전송, opportunistic은 Server가 먼저 전송한다고 가정
→ 두 기대가 충돌
- 구현 버그가 아니라 표준을 그대로 따랐음에도 공격이 가능함 → 설계 결함

4. HTTPS Desynchronization Attacks - Four attack classes



Resource Confusion

- ①=다른 리소스 요청.
- 다른 리소스를 응답으로 받도록 유도 (ex. cat ↔ dog)
다운로드 파일 교체 → JS library 버전, DOMPurify 등 보안 스크립트 무력화

Session Fixation

- ①=공격자 계정 로그인.
- 공격자 계정 세션 쿠키를 클라이언트에 고정
→ CUPS에는 anti-CSRF 토큰을 고정해 admin 인터페이스 CSRF까지 확장(PoC)

Reflected XSS

- ①=XSS 스크립트 삽입.
- 링크 클릭 없이도 GET을 포함한 임의 method/header의 취약점 이용 가능
- 요청을 원하는대로 만들므로 self-XSS로만 존재하던 공격도 일반 XSS로 격상
- Range 헤더 악용해 페이지에 주석 처리 되어있던 `<script>` 추출해 실행(PoC)

Software-Specific Issue (Apache)

- Apache 업그레이드 요청의 body 처리 방식이 비표준
→ 완전한 HTTP request smuggling으로 확장
- HttpOnly/Secure 쿠키까지 유출 가능

4. HTTPS Desynchronization Attacks - HTTP request smuggling

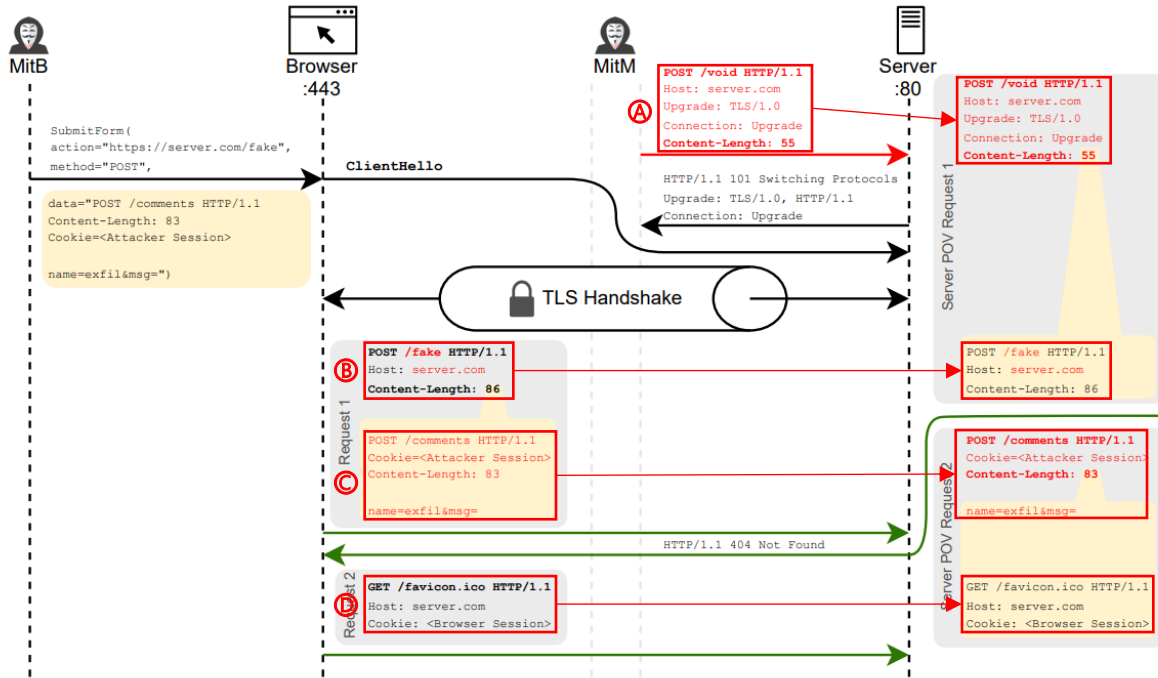


Figure 5: Sketch of an exploit for the HTTP POST behavior of Apache/2.4.62 (without padding). When upgrading a plain

Resource Confusion

- ①=다른 리소스 요청.
- 다른 리소스를 응답으로 받도록 유도 (ex. cat ↔ dog)
다운로드 파일 교체 → JS library 버전, DOMPurify 등 보안 스크립트 무력화

Session Fixation

- ①=공격자 계정 로그인.
- 공격자 계정 세션 쿠키를 클라이언트에 고정
→ CUPS에는 anti-CSRF 토큰을 고정해 admin 인터페이스 CSRF까지 확장(PoC)

Reflected XSS

- ①=XSS 스크립트 삽입.
- 링크 클릭 없이도 GET을 포함한 임의 method/header의 취약점 이용 가능
- 요청을 원하는대로 만들므로 self-XSS로만 존재하던 공격도 일반 XSS로 격상
- Range 헤더 악용해 페이지에 주석 처리 되어있던 <script> 추출해 실행(PoC)

Software-Specific Issue (Apache)

- Apache upgrade 요청의 body 처리 방식이 비표준
헤더까지만 평문으로 받고, 바디는 TLS 이후 암호문으로 받으려고 함
→ HTTP request smuggling으로 확장, HttpOnly / Secure 우회도 가능

- ① MitB의 hidden form이 ②+③을 자동 제출하게 함 → Browser가 ClientHello 전송
- ② MitM이 ClientHello를 delay시키고 Server에 ④를 전송해 opportunistic 개시, Server의 수락과 MitM으로 TLS handshake 정상 수행
- ③ Browser가 ②+③을 TLS로 전송
 - Server는 ②만 Request 1로 인식해 /void의 body로 흡수 → /void는 존재하지 않으므로 404 응답
 - ③은 새 요청으로 재해석되고, 83bytes 중 남은 bytes를 받기 위해 열린 상태로 대기
- ④ Browser가 같은 TLS 연결을 재사용해 ⑤ 전송
 - ⑤가 ③ 뒤에 이어 붙으면서 흡수 → 최종적으로 피해자의 진짜 세션 쿠키가 댓글 내용에 그대로 노출

5. Experiments and Evaluation - IPv4-wide Scan

HTTP Opportunistic TLS Scan (35 ports, 327.7M IP)

구분	호스트 수
HTTP 101 응답 (업그레이드 시도)	36,788
TLS Upgrade 헤더 포함	33,274
실제 TLS 핸드셰이크 응답	20,121
→ 유효 도메인 추출	5,872
→ 동일 도메인 HTTPS(443) 지원	2,268 (38.6%)
→ 인증서 공유 (공격 가능)	539 (9.2%)

FTP / IMAP / POP3 / SMTP Scan (susceptible hosts)

Protocol	Implicit	Opportunistic	Susceptible
FTP	115,799	2,083,439	219,200
IMAP	1,778,397	1,679,746	1,473,974
POP3	1,493,423	1,327,032	1,139,443
SMTP	969,709	420,927	243,153

- 5개 프로토콜·43개 포트에서 **3,000,000+ 호스트가 implicit-opportunistic TLS를 동시 지원**
→ 이론적 취약점이 아니라 실제 배포 규모가 큰 문제
- 다만 HTTP은 **실제 공격 가능 조건(도메인·인증서 일치)까지 만족하는 사례는 539건**으로 상대적으로 적음
IMAP/POP3는 절대 다수가 susceptible하지만, O2I여서 exploitable하지 않고 IMAP은 synchronized protocol도 아님

6. Existing Defense Mechanisms

단기 완화책

- **opportunistic 전용 ALPN** 도입: opportunistic TLS에 별도 ALPN 문자열을 부여해 서버가 두 TLS를 구분
- ALPACA 대응과 유사. 다만 클라이언트·서버 양쪽 모두 패치가 필요해 배포 속도가 느림

근본 방어책

- **opportunistic TLS 전면 폐기, implicit TLS로 통일**
- "일부라도 보호하는 게 낫다"는 설계 철학이 "**opportunistic이 implicit을 오염**"시킬 수 있음
- Poddebniak et al.(USENIX Sec'21)의 결론과 같은 맥락

disclosure 이후

- Cyrus IMAP → 기본값을 implicit TLS로 변경
- Apache 2 → RFC 2817(opportunistic HTTP) 지원 자체를 제거
- IETF에도 관련 표준 폐기를 제안

7. Challenges

표준을 준수하는 대상을 공격해야 함

- 구현의 결함을 이용하지 않고 프로토콜 상의 취약점을 발견해야 함
- "같은 프로토콜 식별자 = 같은 상태 기계"라는 암묵적 전제를 깨트렸고, 다른 표준에도 잠재적으로 남아 있을 수 있음을 시사

ALPACA의 대응책(strict ALPN 검증)을 무력화해야 함

- RFC 9325의 strict ALPN/SNI 검증이 이미 표준화되어 ALPACA 공격이 무력화되었음에도 Opossum이 성립

암호를 깨지 않고 순수 중계만으로 공격해야 함

- 인증서 위조나 암호 primitive 공격 없이, TLS 핸드셰이크는 100% 정상적으로 통과시켜야 함 (지난 DNS Cache Poisoning은 PRNG exploit)
- 메시지 내용이 아니라 메시지 timing과 order만 조작해야 함

8. Contributions

Opportunistic + Implicit TLS 동시 지원 시 발생하는 **신규 인증 취약점** 규명

- ALPACA와 다른 근본 원인(핸드셰이크 이후 "누가 먼저 말하는가"의 차이)에서 비롯됨을 정형화

HTTP(RFC 2817)에 대한 **4가지 실제 익스플로잇 방향** 제시

- Resource Confusion / Session Fixation / Reflected XSS / Apache Request Smuggling

암호를 깨지 않고 **순수 중계**만으로 성공함

- ClientHello를 지연시키고, 두 개로 분리된 단방향 채널(client→server, server→client) 중 한쪽만 골라서 지연·차단
- 암호학적으로는 완전히 정상인 handshake 위에서도 confusion을 만들어내는 데 성공

9. Limitations

O2I desync 프로토콜(FTP/IMAP/SMTP/LMTP/NNTP/POP3)은 PoC 수준 **desync 검증만 진행**

이론적 영향과 실질 위험의 갭

- "one peer만 지원해도 공격 가능"은 강력하지만, HTTP에서는 Browser가 지원하지 않으므로 반드시 Server가 지원해야 함
- HTTP에는 "같은 도메인·인증서를 공유하는 implicit/opportunistic 서버 쌍"이라는 추가 조건이 필요
- 스캔에서 얻은 유효 도메인은 5,872건, 공격이 가능한 도메인은 539건(9.2%) — 실제 **exploitable domain**은 많지 않음

Session Fixation, Reflected XSS 등 **PoC**로만 제시한 공격 기법이 많고, 여러 번 **반복 검증하지 않음**

Request Smuggling은 **Apache의 비표준 동작에 의존, MitB 가정이 필요** — 구현 결함 없이 표준 동작 준수라는 가정과 충돌

10. More Interested Papers (in IEEE S&P'26, USENIX'26)

- 다음 분석 예정 논문

1. 인증 프로토콜 설계 결함 이용 공격

Title	Conference	Content
Analyzing the WebRTC Ecosystem and Breaking Authentication in DTLS-SRTP	USENIX'26	DTLS-SRTP 인증 우회 취약점을 자동 탐지, 24개 실 서비스에서 검증
Audience Injection Attacks: A New Class of Attacks on Web-Based Authorization and Authentication Standards	IEEE S&P'26	OAuth/OIDC의 audience 검증 결함으로 계정 탈취 가능한 신규 공격군

2. 프로토콜 레벨 자원 고갈 이용 DoS

Title	Conference	Content
MadeYouReset: Exploiting HTTP/2 Server-Side Resets for Large-Scale DoS	IEEE S&P'26	HTTP/2 서버의 RST_STREAM이 백엔드 처리를 취소하지 못하는 결함 이용하는 DoS
TED: Abusing Tunnel Hosts and IPv6 Extension Headers for Pulsing DoS Attacks	USENIX'26	IPv6 터널의 Extension Header 처리 결함으로 지속 분산 트래픽을 지연시간 조작으로 특정 순간에 집중시켜 순간 폭증시킴
Bridges to Self: Silent Web-to-App Tracking on Mobile via Localhost	USENIX'26	Localhost/mDNS/WebRTC로 브라우저-앱 격리 우회, 웹-앱 계정 연결 트래킹

Thank you

Q&A